

# Business requirements — merchant risk quality incident

Document: **brd-psp**

Product: four-hour workshop payment-quality slice

Owner: the learner

Merchant under review: **m-007** (NovaPay)

This is the ticket. Encode it into contracts in module 01. Do not implement the seven-entity advanced reference. The machine specification is [docs/specs/psp-payment.md](#).

## 1. Ticket

Risk operations cannot explain rejected payment events. Merchant NovaPay (**m-007**) received a confirmed chargeback, and the risk score did not move. The working theory is that invalid rows were dropped on ingest, so the desk never saw unauthorized BRL traffic or the late valid chargeback.

The product must keep bad payments visible, explain each rejection, and show that a valid late chargeback changes this merchant from risk 25 (**normal**) to 45 (**elevated**).

## 2. Stakeholders

| Role          | Need                                                                                           |
|---------------|------------------------------------------------------------------------------------------------|
| Risk analyst  | Query rejected rows, read <b>_errors</b> / <b>_warnings</b> , see Gold before and after replay |
| Data engineer | One Lakeflow graph, one <b>dev</b> bundle, inspectable Unity Catalog lineage                   |
| Coding agent  | A scoped plan, contracted entities only, stop at each checkpoint                               |

The learner is the named owner through merge of the module commit and the next green checkpoint.

## 3. In scope

- Four entities only: merchants, orders, transactions, disputes
- Exactly 100,000 transactions, seed **22082026**
- Two batches: 98,791 healthy baseline; 1,209 incident rows
- DQX as the sole domain-quality source; invalid rows retained in quarantine
- One Gold table at merchant grain: **gold\_merchant\_risk**
- One serverless Lakeflow pipeline and one **dev** Databricks Asset Bundle
- Genie Code investigation of Gold and quarantine

## 4. Out of scope

Do not build or import:

- customers
- payment\_instruments
- payouts
- KYB onboarding workflows
- production targets, schedules, classic clusters, or service principals
- a six-table or seven-entity payment platform (customers, cards, payouts, KYB)

Those are not this ticket.

## 5. Business rules

- Authorized currencies: **USD, GBP, CAD, AUD**. **BRL** is unauthorized and must remain as explained quarantine.
- Authorized processors: **stripe, adyen**.
- Never use **ON VIOLATION DROP ROW** on the workshop pipeline.
- Referential integrity for valid rows: orders belong to merchants; transactions belong to orders; disputes belong to transactions.
- Evidence kinds stay distinct: local tests, pipeline dry-run, bundle validation, deployment, and runtime queries.

## 6. Acceptance conditions — eight incidents

Seven conditions are invalid and must stay in quarantine:

- 1 null transaction ID
- 2 duplicate transaction ID
- 3 non-positive amount
- 4 unauthorized BRL currency (1,204 rows)
- 5 unknown processor
- 6 orphan order
- 7 dispute closes before it opens

The eighth condition is valid and held for replay:

- 1 late valid chargeback — Silver accepts it; Gold for **m-007** moves 25 → 45

## 7. Success tests

| Check                          | Expected                                                      |
|--------------------------------|---------------------------------------------------------------|
| Transaction count              | 100,000 across the complete story                             |
| Baseline Gold for <b>m-007</b> | risk <b>25</b> , band <b>normal</b>                           |
| After valid replay             | risk <b>45</b> , band <b>elevated</b>                         |
| Quarantine                     | seven explained causes with <b>_errors</b> / <b>_warnings</b> |
| Deploy                         | <b>dev</b> only, learner Free Edition workspace               |
| Investigation                  | four Genie Code questions with inspected SQL                  |

Label each result local, deployed, live runtime, or fallback. Instructor replay is not the learner's live proof.

## 8. How we will work

Follow [how-we-work.md](#): Own, Plan, Execute, Verify, Review, Lesson. Approve the plan before files change. Approve deploy before any remote mutation. Write a lesson after each checkpoint.